

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE - UFRN
CENTRO DE CIÊNCIAS EXATAS E DA TERRA - CCET
CURSO DE BACHARELADO EM CIENCIA DA COMPUTACAO

TRABALHO DA 3 UNIDADE
ALGORITMO E TESTES COMPUTACIONAIS PARA PROBLEMA REAL
MODELADO POR GRAFO

SERGIO MANUEL ALVES MOREIRA
LEONARDO MACHADO MOREIRA

Docente: Silvia Diniz

NATAL, RN — 2026

1. Introdução

Este trabalho conclui a sequência iniciada na 1 unidade (modelagem do problema como grafo bipartido) e aprofundada na 2 unidade (estado da arte). Aqui, implementamos e testamos computacionalmente o modelo identificado como mais adequado para a gestão de bolsas de sangue: o **Fluxo Máximo em Redes**, resolvido pelo algoritmo de **Edmonds-Karp**.

O objetivo é demonstrar, na prática, como o algoritmo resolve o problema real de alocar um estoque limitado de bolsas de sangue (por tipo ABO/Rh) entre grupos de pacientes com demandas distintas, respeitando as regras de compatibilidade sanguínea.

2. Modelagem do Grafo

O grafo de fluxo foi estruturado em 4 camadas, conforme proposto na 2 unidade:

Camada	Papel	Capacidade da aresta
Fonte (S) -> Oferta	Representa o estoque do hemocentro	Qtd. de bolsas disponíveis
Oferta -> Demanda	Compatibilidade ABO/Rh	"Infinita" (10.000)
Demanda -> Sumidouro (T)	Necessidade do grupo de pacientes	Qtd. de pacientes do grupo

A tabela de compatibilidade implementada segue o sistema ABO/Rh: O- e o doador universal (doa para todos os 8 tipos); AB + e o receptor universal (aceita de todos os 8 tipos). A rede resultante possui **18 nós** e **43 arestas** (8 de S->oferta, 27 de compatibilidade, 8 de demanda->T).

3. Algoritmo Implementado

A implementação foi feita em **Python**, sem bibliotecas externas de grafos, para que toda a lógica do algoritmo fosse explícita e auditável.

3.1. Estrutura de dados

O grafo residual foi representado como um dicionário de dicionários (`{u: {v: capacidade}}`), permitindo consultar e atualizar capacidades em tempo $O(1)$. Para cada aresta real $u \rightarrow v$, uma aresta reversa $v \rightarrow u$ com capacidade 0 e criada automaticamente - mecanismo que permite ao algoritmo desfazer alocações subótimas em iterações futuras.

3.2. Edmonds-Karp (Ford-Fulkerson com BFS)

```
def edmonds_karp(self, fonte, sumidouro):  
    fluxo_total = 0
```

```

while True:
    pai = {} # reiniciado a cada iteracao
    if not self.bfs(fonte, sumidouro, pai):
        break # sem caminho aumentante = fluxo maximo atingido

    # gargalo = menor capacidade residual no caminho encontrado
    gargalo = float('inf')
    v = sumidouro
    while v != fonte:
        u = pai[v]
        gargalo = min(gargalo, self.grafo[u][v])
        v = u

    # atualiza capacidades: direta -= gargalo, reversa += gargalo
    v = sumidouro
    while v != fonte:
        u = pai[v]
        self.grafo[u][v] -= gargalo
        self.grafo[v][u] += gargalo
        v = u

    fluxo_total += gargalo
    return fluxo_total

```

A cada iteração, uma BFS localiza o caminho mais curto (em número de arestas) da fonte ao sumidouro. O gargalo - menor capacidade residual do caminho - é somado ao fluxo total e as capacidades são atualizadas, incluindo a aresta reversa para permitir realizações futuras.

Complexidade: $O(V \cdot E^2)$. A rede possui $V=18$ nós e $E=43$ arestas, resultando em limite superior de $O(18 \times 43^2) = O(33.282)$ operações. Na prática, o algoritmo convergiu em apenas 12 iterações no cenário normal - muito abaixo do pior caso teórico.

3.3. Corte mínimo (detecção de gargalo)

Quando o fluxo máximo não cobre toda a demanda, o algoritmo localiza o **corte mínimo**: após a última BFS (que falha em alcançar o sumidouro), os nós ainda alcançáveis a partir da fonte formam o lado S do corte. As arestas saturadas que cruzam para o lado T indicam qual tipo sanguíneo representa o gargalo - informação diretamente aplicável a campanhas de doação direcionadas. O Teorema Max-Flow Min-Cut garante que o valor do fluxo máximo é sempre igual a capacidade total dessas arestas.

4. Testes Computacionais

Foram executados dois cenários para validar o comportamento do algoritmo em situações distintas de estoque.

4.1. Cenário 1 - Estoque Normal

Estoque de **250 bolsas** distribuídas conforme a proporção populacional brasileira (O+ é o tipo mais comum, ~36%). Demanda de **197 pacientes** distribuídos nos 8 grupos sanguíneos.

Tipo	Estoque (bolsas)	Tipo	Demanda (pacientes)
O-	20	O-	12
O+	80	O+	60
A-	15	A-	10
A+	70	A+	55
B-	10	B-	8
B+	35	B+	28
AB-	5	AB-	4
AB+	15	AB+	20
Total	250	Total	197

Resultado: o algoritmo encontrou **12 caminhos aumentantes** e atingiu fluxo máximo de **197**, igual a demanda total - ou seja, **100% dos pacientes foram atendidos**.

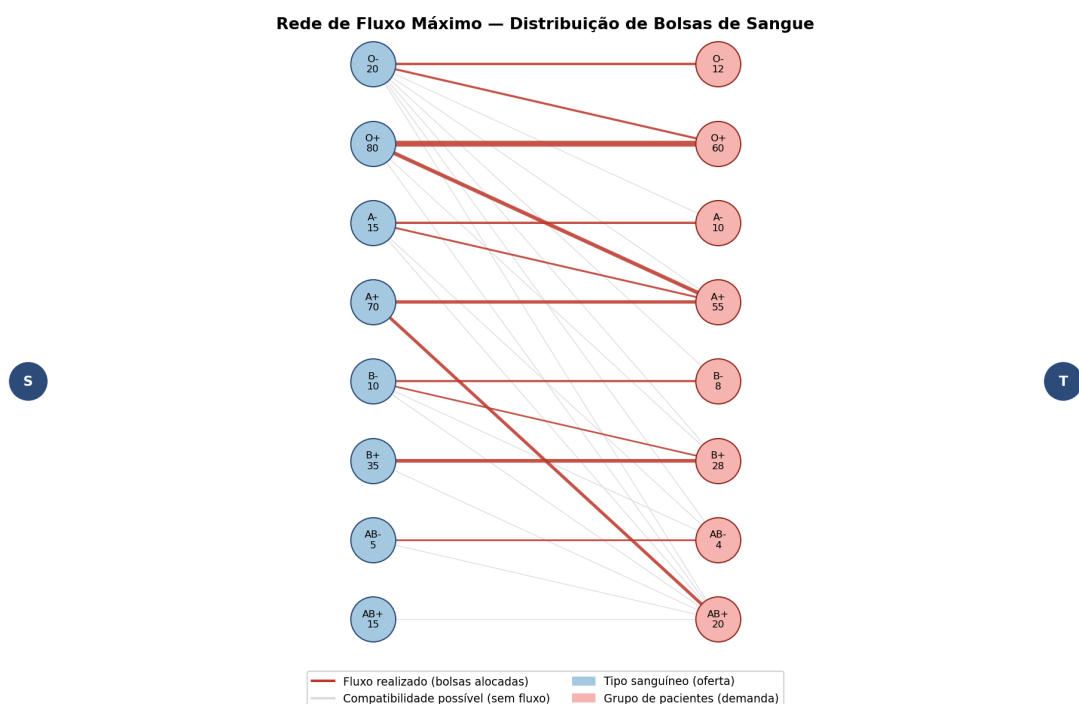
#	Caminho aumentante	Fluxo (bolsas)
01	S -> O- -> Pac_O- -> T	12
02	S -> O- -> Pac_O+ -> T	8
03	S -> O+ -> Pac_O+ -> T	52
04	S -> O+ -> Pac_A+ -> T	28
05	S -> A- -> Pac_A- -> T	10
06	S -> A- -> Pac_A+ -> T	5
07	S -> A+ -> Pac_A+ -> T	22
08	S -> A+ -> Pac_AB+ -> T	20
09	S -> B- -> Pac_B- -> T	8
10	S -> B- -> Pac_B+ -> T	2
11	S -> B+ -> Pac_B+ -> T	26

12	S -> AB- -> Pac_AB- -> T	4
----	--------------------------	---

Relatório de alocação (bolsas enviadas por rota):

Rota (doador -> receptor)	Bolsas alocadas
A+ -> A+	22
A+ -> AB+	20
A- -> A+	5
A- -> A-	10
AB- -> AB-	4
B+ -> B+	26
B- -> B+	2
B- -> B-	8
O+ -> A+	28
O+ -> O+	52
O- -> O+	8
O- -> O-	12

O relatório evidencia o papel do **doador universal O-**: parte de seu estoque foi direcionada para pacientes O+ (8 bolsas), demonstrando como o algoritmo otimiza automaticamente o uso do tipo mais versátil.



4.2. Cenário 2 - Escassez do Tipo O (doador universal)

O estoque de O- foi reduzido de 20 para **3 bolsas** e o de O+ de 80 para **15 bolsas**, simulando um período de baixa captação do tipo sanguíneo mais demandado (ex.: férias, baixa doação sazonal).

Tipo	Estoque normal	Estoque escassez	Reducao
O-	20	3	-85%
O+	80	15	-81%
A-	15	15	—
A+	70	70	—
B-	10	10	—
B+	35	35	—
AB-	5	5	—
AB+	15	15	—

Resultado: O fluxo máximo cai para **143** de uma demanda de **197**, deixando **54 pacientes sem atendimento**. O algoritmo identificou, via corte mínimo, que o gargalo está nos tipos **O- e O+**.

#	Caminho aumentante	Fluxo (bolsas)
01	S -> O- -> Pac_O- -> T	3
02	S -> O+ -> Pac_O+ -> T	15
03	S -> A- -> Pac_A- -> T	10
04	S -> A- -> Pac_A+ -> T	5
05	S -> A+ -> Pac_A+ -> T	50
06	S -> A+ -> Pac_AB+ -> T	20
07	S -> B- -> Pac_B- -> T	8
08	S -> B- -> Pac_B+ -> T	2
09	S -> B+ -> Pac_B+ -> T	26
10	S -> AB- -> Pac_AB- -> T	4

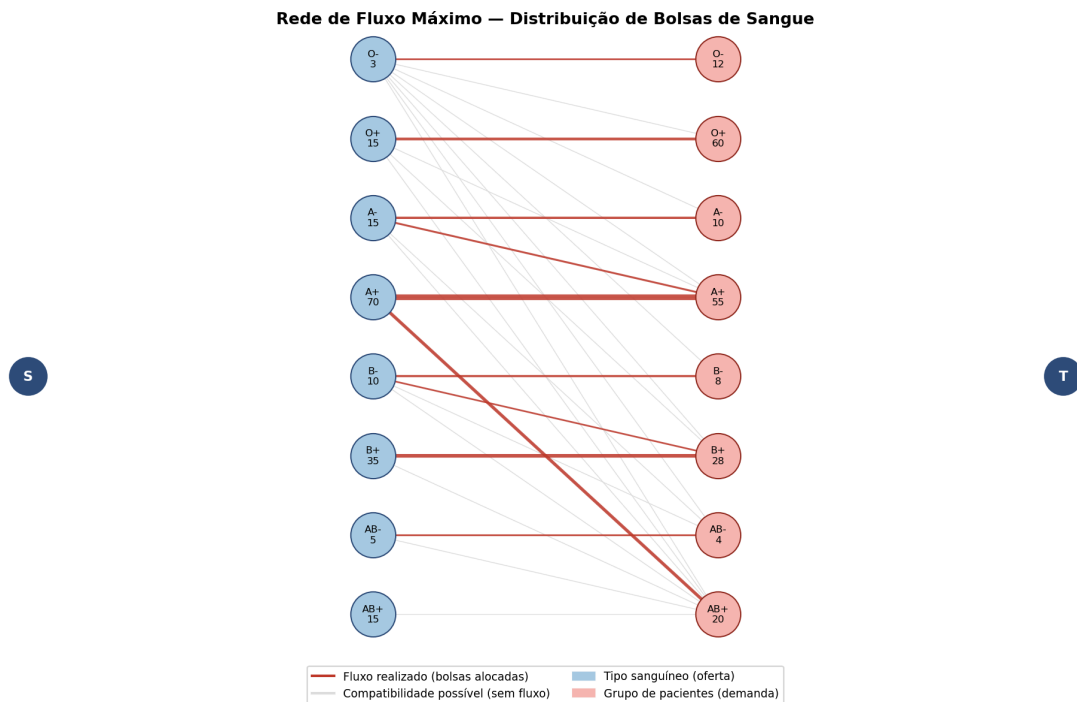
[RESULTADO FINAL - CENARIO 2]

Demanda total : 197 pacientes
 Fluxo maximo : 143 pacientes atendidos
 Pacientes nao atendidos : 54

Corte minimo (gargalo do sistema):

Estoque insuficiente de: O-
Estoque insuficiente de: O+

Esse resultado confirma a utilidade do **Teorema do Corte Mínimo (Max-Flow Min-Cut)**: o sistema não apenas informa que há déficit, mas aponta exatamente qual recurso precisa ser reforçado - informação diretamente aplicável a campanhas de doação direcionadas.



5. Discussão dos Resultados

Os testes confirmam, na prática, a limitação identificada na 1 unidade: um modelo de emparelhamento simples não distinguiria entre os dois cenários, pois não representa quantidade de estoque. O modelo de fluxo máximo captura corretamente:

- A diferença entre 'compatível' e 'disponível em quantidade suficiente';
- O papel do doador universal (O-) como recurso de flexibilização do sistema;
- A identificação automática do gargalo quando a demanda não pode ser plenamente atendida.

6. Conclusão

A implementação do algoritmo de Edmonds-Karp validou, na prática, a escolha de modelo defendida na 2 unidade: o fluxo máximo em redes e a abordagem correta para o problema de distribuição de bolsas de sangue, pois incorpora a restrição de estoque que o emparelhamento bipartido da 1 unidade não contemplava.

Os testes em dois cenários demonstraram que o algoritmo não apenas calcula a alocação ótima, mas também diagnostica gargalos por meio do corte mínimo — capacidade analítica com aplicação direta em hemocentros reais.

7. Referências

As referências bibliográficas completas (Roth et al. 2005; Dillon, Oliveira e Abbasi 2017; Manlove e O'Malley 2015; entre outras) constam no documento da 2 unidade, que fundamentou teoricamente a escolha do modelo implementado neste trabalho.

Anexo — Instruções de Execução

O código completo está disponível no arquivo **blood_flow.py**, entregue em conjunto com este relatório. Para executar:

```
python3 blood_flow.py
```

O programa não depende de bibliotecas externas para o algoritmo (apenas `collections.deque`, da biblioteca padrão do Python). A geração dos gráficos depende de **matplotlib**:

```
pip install matplotlib
```